

Web Technologies Assignment

1. Write HTML5 features with examples and Explain CSS Box Model with diagram.

HTML5 introduced features that reduced the need for external plugins (like Flash) and made code more readable.

A. Semantic Elements

Instead of using `<div>` for everything, HTML5 uses descriptive tags that tell the browser exactly what the content is.

Example: `<header>`, `<footer>`, `<article>`, `<section>`, and `<nav>`.

B. Native Audio and Video

You can embed media directly into the page without third-party players.

Example:

HTML

```
<video width="320" height="240" controls>
<source src="movie.mp4" type="video/mp4">
Your browser does not support the video tag.
</video>
```

C. New Form Input Types

HTML5 added specific types for better mobile keyboards and built-in validation.

Example: `<input type="email">`, `<input type="date">`, `<input type="range">`, and `<input type="color">`.

D. Canvas API

The `<canvas>` element allows for dynamic, scriptable rendering of 2D shapes and bitmap images (often used for games).

Example:

HTML

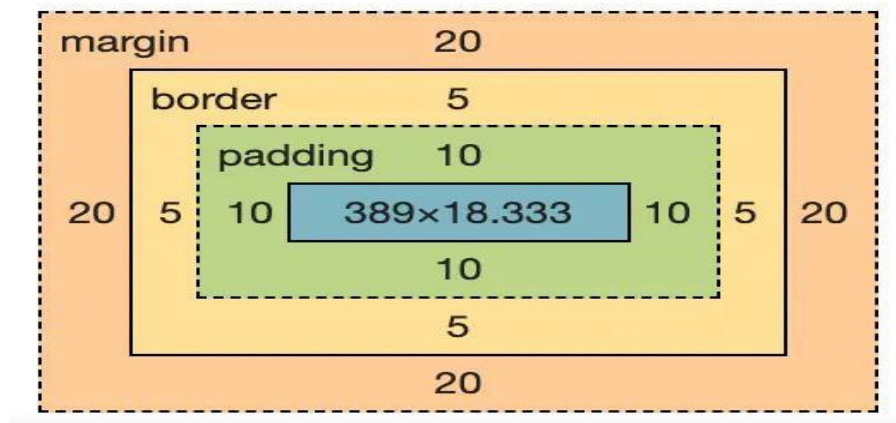
```
<canvas id="myCanvas" width="200" height="100"
style="border:1px solid #000;"></canvas>
```

The CSS Box Model

The CSS Box Model is essentially a box that wraps around every HTML element. It consists of four distinct layers: **Content**, **Padding**, **Border**, and **Margin**.

The Four Layers Explained

Layer	Description
Content	The actual text or image inside the element. Its size is controlled by width and height.
Padding	The transparent area inside the border. It creates space between the content and the border.
Border	The line that goes around the padding and content. You can style its thickness, color, and type.
Margin	The transparent area outside the border. It creates space between this element and its neighbors.



2. Write JavaScript functions, loop and conditions with examples.

JavaScript: Functions, Conditions, and Loops

1. Functions

Functions are reusable blocks of code. They help in modularizing the program.

- **Declaration:** Uses the function keyword.
- **Arrow Function:** A concise modern syntax (ES6).

JavaScript

// Function Declaration

```
function multiply(a, b) {  
    return a * b;  
}
```

//Arrow Function

```
const square = (x) => x * x;
```

2. Conditional Statements

Conditions control the flow of execution based on boolean logic.

If-Else: Executes code if a condition is true.

Switch: Selects one of many code blocks to be executed.

JavaScript

```
let age = 20;
```

```
if(age >= 18) {  
    console.log("Eligible to vote");  
} else {  
    console.log("Not eligible");  
}
```

3. Loops

Loops are used to execute a block of code repeatedly.

- **For Loop:** Used when the number of iterations is known.
- **While Loop:** Continues as long as a condition remains true.

JavaScript

//For Loop

```
for(let i = 1; i <= 3; i++) {  
    console.log("Count: " + i);
```

//While Loop

```
let j = 0;  
while (j < 3) {  
    console.log(j);  
    j++;  
}
```

3. Explain Promises, callbacks and async/await in detail.

Callbacks

- A **function passed as an argument** to another function.
- Used for async tasks.

Example:

```
setTimeout(() => {  
  console.log("Hello");  
}, 1000);
```

Problem: **Callback Hell** (too many nested callbacks → messy code)

Promises

- Object representing **future result** of an async operation.
- States: **Pending, Resolved, Rejected**

Example:

```
let p = new Promise((resolve, reject) => {  
  resolve("Done");  
});  
p.then(res => console.log(res))  
.catch(err => console.log(err));
```

Solves callback hell using **.then() chaining**

Async / Await

- Modern way to handle promises.
- Makes code look **simple and synchronous**

Example:

```
async function getData() {  
  try {  
    let res = await fetchData();  
    console.log(res);  
  } catch (err) {  
    console.log(err);  
  }  
}
```

4. Create and design a responsive landing page using HTML5 and CSS Flexbox.

```
<!DOCTYPE html>

<html> <head>

<meta name="viewport" content="width=device-width, initial-
scale=1"> <style>

body{margin:0;font-family:sans-serif}

.nav,.hero,.f{display:flex;flex-wrap:wrap;padding:10px}

.nav{justify-content:space-between;background:#333;color:#fff}

.hero div,.f div{flex:1;min-width:150px;padding:10px}

@media(max-width:600px){.nav,.hero{flex-direction:column}}

</style>

</head>

<body>

<div class="nav">MySite <div>Home About</div></div>

<div class="hero">

  <div><h2>Welcome</h2></div>

  <div></div>

</div>

<div class="f">

  <div>Fast</div><div>Responsive</div><div>Easy</div>

</div>

</body></html>
```

5. Write a JS program to demonstrate arrays and objects.

// Array example

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log("Fruits:", fruits);
```

//Object example

```
let student = {  
  name: "Rahul",  
  age: 21,  
  course: "B.Tech"  
};
```

```
console.log("Student:", student);
```

// Update values

```
student.name = "Amit";  
student.course = "MBA";
```

```
console.log("Updated Student:", student);
```

6. Explain DOM manipulation with examples.

DOM (Document Object Model) allows JavaScript to **access and change HTML elements dynamically**.

You can:

- Change content
- Change styles
- Add/remove elements

1. Selecting Elements

```
let heading = document.getElementById("title");  
let items = document.getElementsByClassName("item");
```

2. Changing Content

```
document.getElementById("title").innerHTML = "New Title";
```

3. Changing Style

```
document.getElementById("title").style.color = "red";
```

4. Adding Elements

```
let p = document.createElement("p");  
p.innerHTML = "New Paragraph";  
document.body.appendChild(p);
```

5. Event Handling

```
function changeText() {  
  document.getElementById("title").innerHTML = "Clicked!";  
}
```

HTML:

```
<button onclick="changeText()">Click Me</button>
```

7.Explain middleware with example in Node.js.

Middleware is a **function that runs between request and response** in a Node.js app (usually with Express).

It can:

- Modify request/response
- Execute code
- End request or call next middleware

Syntax

```
function middleware(req, res, next) {  
  // do something  
  next(); // move to next middleware  
}
```

Example

```
const express = require("express");  
const app = express();  
  
//Middleware  
app.use((req, res, next) => {  
  console.log("Middleware executed");  
  next();  
});  
  
//Route  
app.get("/", (req, res) => {  
  res.send("Hello World");  
});  
  
app.listen(3000);
```

8. Explain JWT authentication with example.

JWT (**JSON Web Token**) is a method used to **securely authenticate users**.

It is a **token** sent by the server after login and used for future requests.

Structure of JWT

A JWT has **3 parts**:

1. Header
2. Payload (data like user info)
3. Signature

Format:

header.payload.signature

How JWT Works

1. User logs in
2. Server verifies credentials
3. Server sends JWT token
4. Client stores token
5. Token is sent with each request
6. Server verifies token

Example (Node.js)

```
const express = require("express");  
const jwt = require("jsonwebtoken");
```

```
const app = express();  
app.use(express.json());
```

```
//Login route
app.post("/login", (req, res) => {
  const user = { id: 1, name: "Vivek" };

  const token = jwt.sign(user, "secretKey");
  res.json({ token });
});

//Protected route
app.get("/profile", (req, res) => {
  const token = req.headers["authorization"];

  jwt.verify(token, "secretKey", (err, decoded) => {
    if(err) return res.send("Invalid Token");
    res.json({ message: "Access Granted", user: decoded });
  });
});

app.listen(3000);
```

9. Write a program to manipulate DOM elements.

```
<!DOCTYPE html>

<html>

<body>

<h2 id="title">Hello</h2>

<button onclick="change()">Click Me</button>

<script>

function change() {

    // change text
    document.getElementById("title").innerHTML = "Welcome";


    // change style
    document.getElementById("title").style.color = "blue";


    // create new element
    let p = document.createElement("p");
    p.innerHTML = "New Paragraph Added";


    // add to body
    document.body.appendChild(p);
}

</script>

</body>

</html>
```

10. Implement file handling using Node.js File System module.

```
const fs = require("fs");

// Write to file
fs.writeFile("example.txt", "Hello World", (err) => {

  if (err) throw err;
  console.log("File created");
  // Read file
  fs.readFile("example.txt", "utf8", (err, data) => {
    if (err) throw err;
    console.log("Content:", data);
    // Append data
    fs.appendFile("example.txt", "\nNew Line Added", (err) => {
      if (err) throw err;
      console.log("Data appended");
      // Delete file
      fs.unlink("example.txt", (err) => {
        if (err) throw err;
        console.log("File deleted");
      });
    });
  });
});
```

writeFile() -> create/write file

readFile() -> read file

appendFile() -> add content

unlink() -> delete file

11. Describe REST API development using Express.

REST API using Express

REST API is used to **perform CRUD operations using HTTP methods.**

- **GET** -> read data
- **POST** -> create
- **PUT** -> update
- **DELETE** -> delete

Example

```
const express = require("express");
const app = express();

app.get("/", (req, res) => res.send("GET"));
app.post("/", (req, res) => res.send("POST"));
app.put("/", (req, res) => res.send("PUT"));
app.delete("/", (req, res) => res.send("DELETE"));

app.listen(3000);
```

12. Explain React component lifecycle.

It is the **different stages a React component goes through** from creation to removal.

3 Main Phases

1. Mounting (Creation)

Component is created and added to DOM.

- constructor()
- render()
- componentDidMount()

2. Updating

Occurs when state or props change.

- shouldComponentUpdate()
- render()
- componentDidUpdate()

3. Unmounting

Component is removed from DOM.

- componentWillUnmount()

Example

```
class MyComponent extends React.Component {
  componentDidMount() {
    console.log("Mounted");
  }
  componentDidUpdate() {
    console.log("Updated");
  }
  componentWillUnmount() {
    console.log("Removed");
  }
  render() {
    return <h1>Hello</h1>;
  }
}
```

13. Compare functional and class components.

Feature	Functional Component	Class Component
Syntax	Simple function	ES6 class
Code Length	Short	Longer
State	Uses Hooks (useState)	Uses this.state
Lifecycle	Hooks (useEffect)	Lifecycle methods
Performance	Faster	Slightly slower
Usage	Modern (preferred)	Older approach

14. Implement React Hooks (useState, useEffect).

```
import React, { useState, useEffect } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  //useEffect runs after render
  useEffect(() => {
    console.log("Count changed:", count);
  },[count]);

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}
```

```
export default Counter;
```

What it shows

- useState -> manage state
- useEffect -> run side effects (after render)

15. Build a React component with props and state.

```
import React, { useState } from "react";

function Counter(props) {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h3>Hello {props.name}</h3>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

```
export default Counter;
```

Usage

```
<Counter name="Vivek" />
```

16. Create login/signup form using react.

```
import React, { useState } from "react";

function AuthForm() {
  const [isLogin, setIsLogin] = useState(true);
  const [form, setForm] = useState({ name: "", email: "", password: "" });

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(isLogin ? "Login Successful" : "Signup Successful");
  };

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h2>{isLogin ? "Login" : "Signup"}</h2>

      <form onSubmit={handleSubmit}>
        {!isLogin && (
          <input
```

```
    type="text"
    name="name"
    placeholder="Name"
    onChange={handleChange}
  />
})
<br/>
```

```
<input
  type="email"
  name="email"
  placeholder="Email"
  onChange={handleChange}
/>
<br/>
```

```
<input
  type="password"
  name="password"
  placeholder="Password"
  onChange={handleChange}
/>
<br/>
<button type="submit">
```

```

        {isLogin ? "Login" : "Signup"}
      </button>
    </form>

    <onClick={() => setIsLogin(!isLogin)} style={{ cursor: "pointer" }}>
      {isLogin ? "Create account?" : "Already have an account?"}
    </p>
  </div>
);
}

export default AuthForm;

```

Key Points

- useState : manages form + mode (login/signup)
- Conditional rendering : shows **name field only in signup**
- Single form handles both login & signup
- Toggle using setIsLogin()

17. Describe MongoDB Setup.

MongoDB is a **NoSQL database** that stores data in **JSON-like documents** instead of tables.

Steps to Install MongoDB

1. Download MongoDB

- Go to: MongoDB Inc. official website
- Download **MongoDB Community Server**

2. Install MongoDB

- Run the installer
- Choose **Complete Setup**
- Install **MongoDB Compass** (GUI tool)

3. Start MongoDB Server

- MongoDB runs as a service automatically
- Or start manually:

```
mongod
```

4. Open Mongo Shell

```
mongosh
```

5. Create Database & Collection

```
use myDatabase
```

```
db.users.insertOne({  
  name: "Mohammed",  
  age: 21  
})
```

6. View Data

```
db.users.find()
```

Optional: Use MongoDB Compass

- GUI tool to manage database visually
- Connect using: mongodb://localhost:27017

Key Points

- NoSQL :- no tables, uses collections
- Data stored in **documents (JSON/BSON)**
- Easy to scale and flexible schema
- Used with Node.js (MERN stack)

18. Create a Database, insert data and perform CRUD operations.

1.Create Database

```
use studentDB
```

Insert Data (Create)

```
db.students.insertOne({ name: "Mohammed", age: 21 })
```

2. Read Data

```
db.students.find()
```

3 .Update Data

```
db.students.updateOne(  
  { name: "Mohammed" },  
  { $set: { age: 23 } }  
)
```

4 Delete Data

```
db.students.deleteOne({ name: "Mohammed"  
})
```

19. Connect MongoDB with Node.js, Create schema and model, Perform CRUD operations using Mongoose

Tools

- MongoDB
- Node.js
- Mongosh

1)Connect to MongoDB

```
const mongoose = require("mongoose");
```

```
mongoose.connect("mongodb://127.0.0.1:27017/collegeDB")  
.then(() => console.log("Connected"));
```

2)Schema & Model

```
const studentSchema = new mongoose.Schema({  
  name: String,  
  age: Number,  
  course: String  
});
```

```
const Student = mongoose.model("Student", studentSchema);
```

3) CRUD Operations

Create

```
const s1 = new Student({ name: "Ankit", age: 21, course: "BCA" });  
s1.save();
```

Read

```
Student.find().then(data => console.log(data));
```

Update

```
Student.updateOne(  
  { name: "Ankit" },  
  { $set: { age: 23 } }  
);
```

Delete

```
Student.deleteOne({ name: "Ankit" });
```

20. Create two collections:

- **Students**
- **Courses**

And perform Join operation.

Using MongoDB (\$lookup)

1>Create Collections & Insert Data

//Courses collection

```
db.courses.insertMany([
  { _id: 1, course: "BCA" },
  { _id: 2, course: "BBA" }
]);
```

//Students collection

```
db.students.insertMany([
  { name: "Ankit", course_id: 1 },
  { name: "Rahul", course_id: 2 }
]);
```

2.Join Using \$lookup

```
db.students.aggregate([
  {
    $lookup: {
      from: "courses",
      localField: "course_id",
      foreignField: "_id",
      as: "course_details"
    }
  }
]);
```

Output

- Combines **students + course info**
- Similar to SQL JOIN

21. Perform database connectivity with backend, frontend.

Stack

Frontend: React

Backend: Node.js

Database: MongoDB

1Backend (API)

```
const express = require("express");
```

```
const mongoose = require("mongoose");
```

```
const app = express();
```

```
app.use(express.json());
```

```
mongoose.connect("mongodb://127.0.0.1:27017/myDB");
```

```
const User = mongoose.model("User", { name: String });
```

```
app.post("/add", async (req, res) => {
```

```
    await User.create(req.body);
    res.send("Saved");
  });

  app.get("/users", async (req, res) => {
    res.json(await User.find());
  });

  app.listen(5000);
```

2Frontend (React)

```
fetch("http://localhost:5000/add", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Ankit" })
});

fetch("http://localhost:5000/users")
  .then(res => res.json())
  .then(data => console.log(data));
```